
Redes Neurais e Aprendizagem Profunda

— Treino de redes profundas —

Sumário

Training a deep NN isn't a walk in the park!

Vanishing e exploding gradients

Reutilização de camadas pré-treinadas

Otimizadores rápidos

Regularização

Redes profundas

Imagem

- Xception
 - 22.9 M
- VGG16
 - 138.4 M
- ResNet50
 - 25.6M
- EfficientNetB7
 - 66.7 M
- MobileNet
 - 4.3 M

Texto

- BERT
 - base: 110 M, large: 340 M
- DistilBERT
 - 66 M
- ALBERT
 - base: 12 M, large: 18 M
- GPT-3 (OpenAI)
 - 175 B
- MT-NLG (Microsoft e NVIDIA)
 - 530 B
- LLaMA (MetaAI)
 - 1.2 trillion
- BARD (Google)
 - 1.6 trillion

Problemas no treino de redes profundas

- Vanishing/exploding gradients
- Quantidade de dados insuficiente
- Treino demasiado lento
- Risco de sobre-ajustamento

Vanishing & exploding gradients

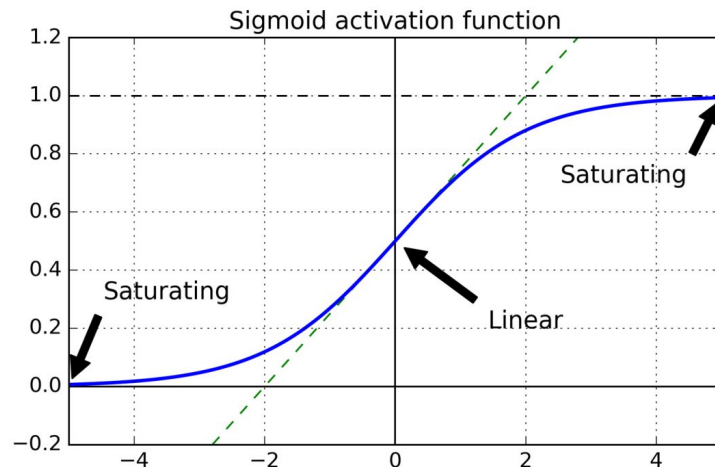
Backpropagation

- Funcionamento
 - propaga o gradiente do erro da camada de saída até à de entrada (para cada parâmetro da rede)
 - atualiza os parâmetros através de um passo da descida do gradiente
- Vanishing gradients
 - gradientes tornam-se cada vez mais pequenos ao progredir para camadas mais baixas
 - os pesos mantêm-se (virtualmente) inalterados
 - o algoritmo não converge para uma boa solução
- Exploding gradients
 - gradientes tornam-se cada vez maiores
 - pesos tornam-se cada vez maiores
 - o algoritmo diverge

Porquê?

- Nos anos 2000 as razões não eram claras...
 - uma razão para o desinteresse nas RNA
- Em 2010 (Glorot & Bengio)
 - problema deve-se à combinação da função de ativação e técnica inicialização dos pesos
 - função ativação: sigmoide
 - inicialização: distribuição normal (média=0, desvio padrão=1)
 - a variância na saída de cada camada muito superior à na entrada

<https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>



- Consequência
 - a função satura nas camadas mais altas
 - derivada muito perto de 0
 - gradiente inexistente
 - qualquer pequeno gradiente é diluído com a progressão da backpropagation para as camadas mais baixas

Inicialização de Glorot

- O sinal deve fluir em ambos os sentidos
 - para a frente quando são feitas as previsões
 - em cada camada a variância das saídas deve ser igual à das entradas
 - para trás quando são propagados os gradientes
 - variância dos gradiente deve ser igual antes e depois de propagar numa camada
- Proposta
 - inicializar os pesos aleatoriamente (dist. normal ou uniforme) de acordo com
 - fan-avg: média(fan-in, fan-out)
 - fan-in: n° entradas de uma camada
 - fan-out: n° de saídas de uma camada (neurónios)

Normal distribution with mean 0 and variance $\sigma^2 = \frac{1}{fan_{avg}}$

uniform distribution between $-r$ and $+r$, with $r = \sqrt{\frac{3}{fan_{avg}}}$

Função de ativação ReLU

- Vantagens

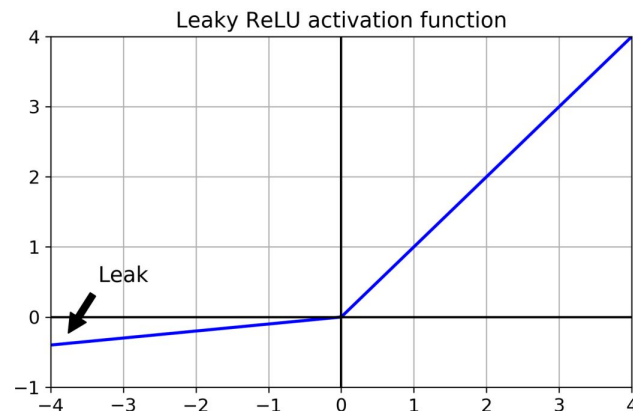
- funciona muito melhor que sigmoide em redes profundas
 - não satura
 - cálculo rápido

- Problema

- *dying* ReLUs
 - neurónios que só geram zeros
 - quando a soma pesada das entradas é negativa para todos os exemplos
 - gradiente é zero
 - descida do gradiente não funciona

Variantes

- leaky ReLU
 - $\max(\alpha z, z)$
 - $\alpha=0.01$ (valor típico)
- RReLU
 - randomized leaky ReLU
 - α : dist. uniforme entre (min, max)
- PReLU
 - parametric leaky ReLU
 - α : aprendido durante o treino



<https://arxiv.org/abs/1505.00853>

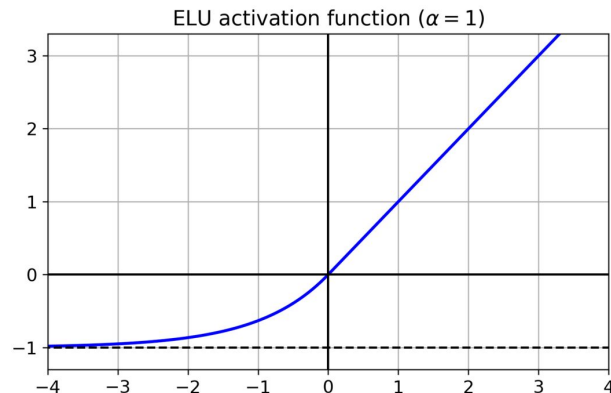
ELU

- Exponential linear unit

$$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

- Características

- negativa quando $z < 0$
 - alivia o problema dos vanishing gradients
- gradiente $\neq 0$ para $z < 0$
 - evita os neurónios "mortos"
- $\alpha=1$
 - função suave em todos os pontos
- converge mais rapidamente
- cálculo mais lento



SELU

- Scaled ELU
- Características
 - auto-normalização (rede composta de camadas densas)
 - a saída de cada camada tende a preservar $\text{media}=0$, $\text{desvio}=1$
 - Supera outras funções
- Condições para auto-normalização
 - atributos de entrada normalizados
 - cada camada escondida inicializada com inicialização LeCun
 - arquitetura sequencial

<https://arxiv.org/abs/1706.02515>

Que função usar?

- SELU > ELU > leaky ReLU > ReLU > tanh > logistic
- ELU
 - quando a arquitetura não permite auto-normalização
- leaky ReLU
 - se latência de execução é importante
- variantes ReLU
 - se houver tempo e poder computacional (testar com validação cruzada)
 - RReLU se a rede estiver a sobre-ajustar
 - PReLU se o conj treino for enorme
- ReLU
 - se a velocidade for a prioridade

Inicialização e funções de ativação

Initialization	Activation functions	σ^2 (Normal)
Glorot	None, tanh, logistic, softmax	$1 / fan_{avg}$
He	ReLU and variants	$2 / fan_{in}$
LeCun	SELU	$1 / fan_{in}$

Batch normalization

- Evita os vanishing/exploding gradients
 - a inicialização e função de ativação reduzem drasticamente o perigo mas não evitam...
- Técnica
 - adicionar uma operação na arquitetura **antes ou depois** da função de ativação de cada camada escondida
 - o modelo aprende os vetores ótimos para escalar e centrar no zero
 - centra no zero e normaliza a entrada
 - altera o resultado usando 2 vetores de parâmetros por camada
 - scale
 - shift
- Estima
 - média e desvio padrão da entrada usando o mini-batch corrente

Funcionamento

- Treino
 - normaliza as entradas
 - re-escala e desvia-as de acordo as estatísticas do batch específico
- Teste
 - utiliza a estimativa das médias e desvios padrão das camadas BN calculada no treino (usando moving averages)
- O treino de cada época é mais lento...
... mas converge mais depressa!
- Funciona como regularizador...
... mas as previsões são mais lentas!
 - pode ser otimizado
- Parâmetros aprendidos (para cada camada BN)
 - backpropagation
 - γ : vetor *scale* de saída
 - β : vetor *shift* de saída
 - estimados usando moving average
 - μ : vetor média de entrada final
 - σ : vetor desvio de entrada final

Keras

- Inicialização de pesos
 - parâmetro `"kernel_initializer"`
 - `he_uniform`
 - `he_normal`
- Função de ativação
 - parâmetro `"activation"`
 - `relu`
 - `elu`
 - `selu`
 - adição de camada
 - `LeakyReLU()`
 - `PReLU()`
- Batch normalization
 - adição de camada
 - `BatchNormalization()`

Onde colocar BN?

BN depois da função de ativação

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation="softmax")
])
```

Layer (type)	Output Shape	Param #
flatten_3 (Flatten)	(None, 784)	0
batch_normalization_v2 (Batch Normalization)	(None, 784)	3136
dense_50 (Dense)	(None, 300)	235500
batch_normalization_v2_1 (Batch Normalization)	(None, 300)	1200
dense_51 (Dense)	(None, 100)	30100
batch_normalization_v2_2 (Batch Normalization)	(None, 100)	400
dense_52 (Dense)	(None, 10)	1010
Total params: 271,346		
Trainable params: 268,978		
Non-trainable params: 2,368		

BN antes da função de ativação

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, kernel_initializer="he_normal", use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("elu"),
    keras.layers.Dense(100, kernel_initializer="he_normal", use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("elu"),
    keras.layers.Dense(10, activation="softmax")
])
```

A melhor opção parece depender da tarefa...

Hiper-parâmetros de BN

- `momentum`
 - para atualizar as moving averages
 - valor próximo de 1
 - ex: 0.9, 0.99, 0.999
 - mais 9s para conj grandes e mini-batches mais pequenos
- `axis`
 - eixo a normalizar
 - default (-1): último eixo
 - exemplo
 - normaliza (*scale* e *shift*) 784 atributos de entrada
 - cada atributo é normalizado baseado na média e desvio padrão calculados sobre todas as instâncias do batch

BN e redes profundas

Utilização

- BN é uma das camadas mais usadas em RP
 - muitas vezes não são representadas nos diagramas

Inicialização de pesos fixed-update

- proposta que dispensa BN atingindo desempenhos equivalentes em tarefas complexas de classificação de imagens

<https://arxiv.org/abs/1901.09321>

Gradient clipping

Técnica

- Limita gradientes durante backpropagation
 - Técnica para evitar exploding gradients
- Usada principalmente em redes recorrentes

Keras

- Parâmetro do otimizador
 - `clipvalue`
 - pode mudar a orientação do gradiente
 - `clipvalue=1`
 - `[0.9, 100.0] -> [0.9, 1]`
 - `clipnorm`
 - preserva a orientação
 - pode eliminar componentes
 - `clipnorm=1`
 - `[0.9, 100.0] -> [0.008999, 0.99995]`

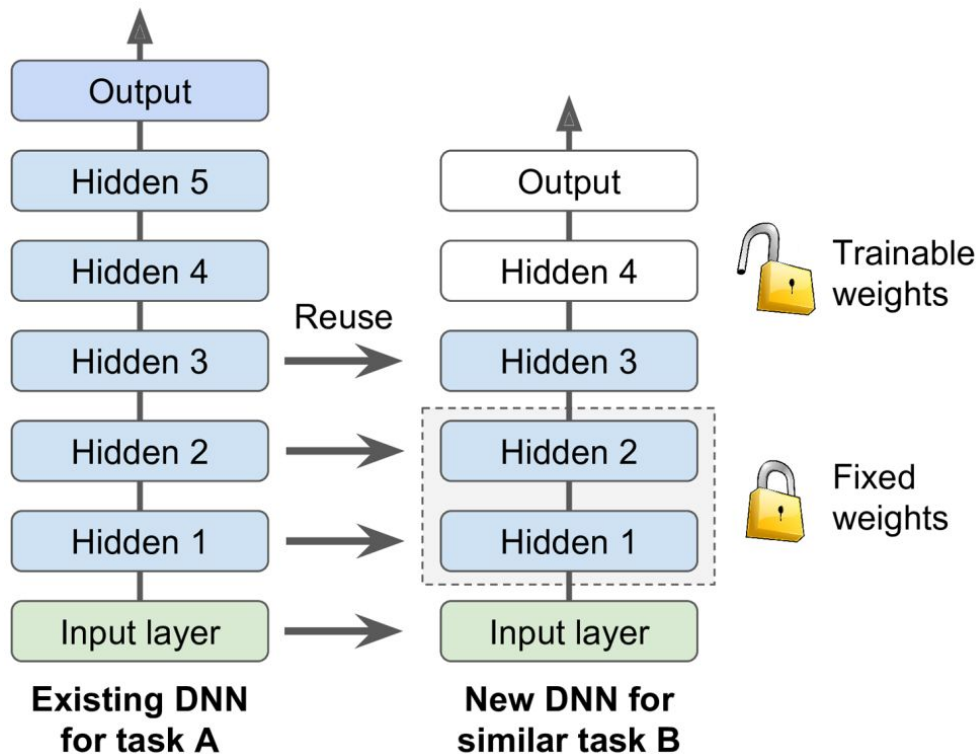
```
optimizer = keras.optimizers.SGD(clipvalue=1.0)
model.compile(loss="mse", optimizer=optimizer)
```

Reutilização de camadas pré-treinadas

Transfer learning

- Ideia
 - Usar uma rede pré-treinada para uma tarefa similar reutilizando as camadas mais baixas da rede
- Vantagens
 - exige muito menos tempo de treino
 - requer muito menos dados
- Exemplo
 - usar uma rede que distingue **100 categorias** (entre animais, plantas, veículos e outros objetos) para treinar distinção entre **tipos de veículos específicos**

Reutilização de camadas pré-treinadas



Que camadas manter?

- Substituir a camada de saída
 - normalmente não é útil
 - pode não ter o mesmo nº de neurónios
- Camadas escondidas superiores
 - menor probabilidade de serem úteis
 - características de alto nível da nova tarefa podem ser diferentes de forma significativa às usadas na tarefa original
- Quanto mais similares, mais camadas reutilizar
 - para tarefas muito similares é possível substituir apenas a camada de saída (e manter as escondidas)

Processo

1. **Congelar** todas as camadas escondidas, treinar o modelo e ver o desempenho
 - indicar que os pesos não são treináveis, para a descida do gradiente não os modificar
2. **Descongelar uma ou duas camadas** superiores e ver qual o aumento do desempenho
 - diminuir a taxa de aprendizagem para evitar "estragar" os pesos afinados
 - quanto mais dados existirem mais camadas podem ser descongeladas
3. Se o desempenho for baixo e
 - existirem poucos dados: **remover camada(s)** superior(es) e congelar as camadas restantes
 - iterar até encontrar o nº adequado de camadas
 - existirem muitos dados: **substituir a(s) camada(s)** superiores (em vez de as remover) ou até adicionar novas camadas

Exemplo

Problema

- tarefa A: modelo existente
 - Fashion MNIST com 8 classes (sem `sandal` e `shirt`)
- tarefa B: modelo a construir
 - distinguir entre `sandal` e `shirt`, conjunto de 200 exemplos

Transfer learning

- modelo A na tarefa B
 - exatidão: 97.2%
- pesos das camadas escondidas afinados
 - exatidão: 99.25%
- **alerta!**
 - desempenho conseguido depois de muitas variações!
 - resultados publicados em artigos podem ser enganadores por não indicarem o nº de experiências realizadas...

Keras

```
model_A = keras.models.load_model("my_model_A.h5")
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd",
                     metrics=["accuracy"])

history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                          validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = keras.optimizers.SGD(lr=1e-4) # the default lr is 1e-2
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                          validation_data=(X_valid_B, y_valid_B))
```

substituir camada de saída

congelar as camadas escondidas

compilar modelo

treinar por algumas épocas

descongelar camadas

continuar o treino com taxa
aprendizagem menor

Pré-treino não supervisionado

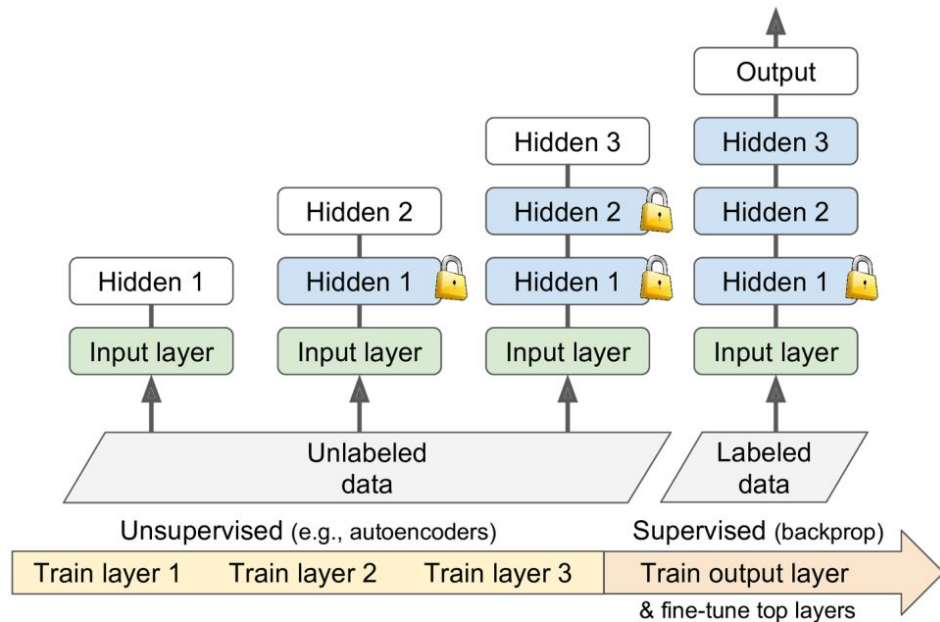
- Problema
 - tarefa complexa
 - poucos dados etiquetados e não existência de modelo para tarefa similar
 - muitos dados não etiquetados
- Ideia
 - **usar dados não etiquetados** e treinar um modelo não supervisionado e reutilizar as camadas inferiores para afinar um modelo supervisionado

Evolução

- 2006
 - Arquiteturas
 - RBM (restricted Boltzmann machines)
 - Técnica
 - Greedy layer-wise pretraining
 - renascimento das redes neuronais e sucesso da aprendizagem profunda
- 2010
 - Arquiteturas
 - Autoencoders, GAN (generative adversarial network)
 - Técnica
 - treino do modelo não supervisionado completo

Greedy layer-wise pretraining

1. treinar modelo não supervisionado com uma única camada
2. congelar camada e adicionar uma no topo
3. voltar a treinar
4. congelar camada e adicionar uma no topo
5. ...
6. treinar modelo supervisionado



Pré-treino em tarefa auxiliar

- Problema
 - tarefa complexa
 - poucos dados etiquetados e não existência de modelo para tarefa similar
- Ideia
 - **treinar uma tarefa auxiliar** onde é possível obter ou gerar muitos dados etiquetados e reutilizar as camadas inferiores para a tarefa

Exemplo 1

- Tarefa (visão computacional)
 - reconhecimento de faces com poucas imagens para cada pessoa
- Tarefa auxiliar
 - detetar se duas imagens são da mesma pessoa
 - fácil recolher muitas imagens da web para esta tarefa...
- Processo
 - o modelo detetor irá "aprender" características que detetam faces
 - utilizando as camadas inferiores permite um bom classificador de faces com poucas imagens

Exemplo 2

- Tarefa (processamento de língua natural)
 - geração automática de etiquetas a partir de um corpus
- Tarefa auxiliar
 - prever as palavras em falta num texto, mascarando algumas palavras
- Modelo resultante
 - "saberá" muito sobre a língua...
 - poderá ser utilizado para outras tarefas afinando-o com os dados etiquetados

Self-supervised learning

- **geração automática de etiquetas** a partir dos dados; treino posterior de modelo supervisionado a partir dessas etiquetas

Otimizadores rápidos

Gradiente com momento

- Ideia
 - Acelerar os passos da descida usando a noção de momento (1964)
- Descida do gradiente
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$
 - não tem em conta os gradientes anteriores
 - se o gradiente local for pequeno, os passos são pequenos
- Otimização com momento
 1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
 2. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{m}$
 - o gradiente é utilizado para **acelerar o processo** e não a sua velocidade

Parâmetro β

- β : momento
 - introduz "fricção" no processo e previne que o momento se torne demasiado grande
 - $\beta=0$ -> alta fricção
 - $\beta=1$ -> sem fricção
- Velocidade terminal
 - É o valor máximo da atualização dos pesos
 - se o gradiente for contante
$$v_t = \frac{1}{1 - \beta} \eta \nabla_{\theta} J(\theta)$$
 - $\beta=0.9$ -> velocidade de otimização c/ momento é 10 vezes superior (quando comparado com a descida do gradiente normal)

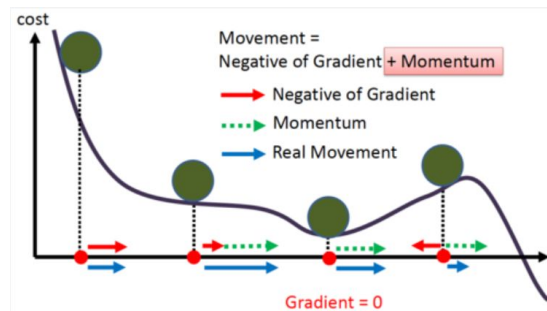
Vantagens e desvantagens

Vantagens

- Permite escapar de planaltos muito mais rapidamente
 - quando as entradas têm escalas muito diferentes (sem BN acontece nas camadas superiores)
 - descida do gradiente: desce a encosta íngreme depressa, mas depois leva muito tempo para descer o "vale"
 - momento: desce o "vale" cada vez mais depressa até chegar ao ótimo
- Ajuda a ultrapassar ótimos locais

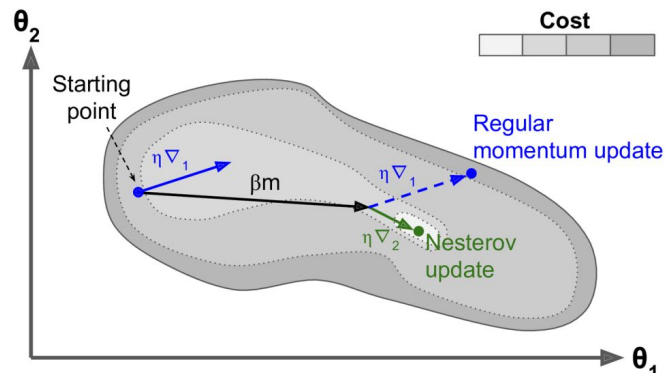
Desvantagens

- pode oscilar um pouco
- mais um parâmetro para afinar
- $\beta=0.9$
 - costuma funcionar bem e ser mais rápido que a descida do gradiente



NAG - Nesterov Accelerated Gradient

- Alteração proposta em 1983
 1. $\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta + \beta \mathbf{m})$
 2. $\theta \leftarrow \theta + \mathbf{m}$
 - mede o gradiente não na posição θ , mas um pouco mais à frente na direção do momento
- Em geral o vetor momento aponta na direção correta
 - o gradiente será um pouco mais preciso se medido um pouco à frente nessa direção
- Otimização significativamente mais rápida que o gradiente com momento
- Ajuda a reduzir as oscilações

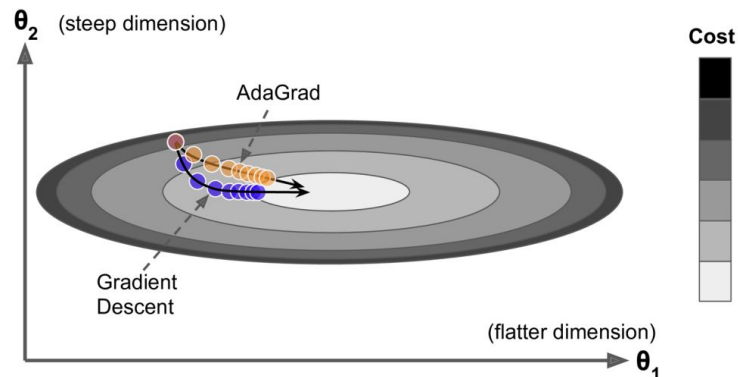


Adagrad

- Alteração proposta em 2011
 1. $\mathbf{s} \leftarrow \mathbf{s} + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
 2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{\mathbf{s} + \varepsilon}$
 - $\mathbf{s}$: acumula os quadrados do gradiente
 - ε : evita a divisão por zero (valor típico: 10^{-10})
 - corrige a direção do gradiente para apontar para o ótimo global mais cedo, reduzindo o vetor de gradiente ao longo das dimensões mais íngremes
- provoca diminuição **adaptativa da taxa aprendizagem**
 - mais rápida para dimensões íngremes
 - mais lenta para dimensões com inclinação suave

<https://jmlr.org/papers/v12/duchi11a.html>

- Vantagens
 - requer menos afinação da taxa de aprendizagem
- Desvantagens
 - muitas vezes **pára demasiado cedo**
 - diminuição muito forte da taxa aprendizagem faz parar a otimização



RMSProp

- Alteração ao Adagrad proposta em 2012

1. $\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

2. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \oslash \sqrt{\mathbf{s} + \varepsilon}$

- β : fator de decréscimo

- acumula apenas os gradientes das iterações mais recentes

- Vantagens

- muito melhor que Adagrad (na maioria dos problemas)

- Desvantagens

- mais um parâmetro para afinar
- $\beta=0.9$ costuma funcionar bem

Adam: adaptive moment estimation

- Proposto em 2014
- Combina ideias do momento e RMSProp

1. $\mathbf{m} \leftarrow \beta_1 \mathbf{m} - (1 - \beta_1) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
2. $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
3. $\widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^T}$
4. $\widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^T}$
5. $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \varepsilon}$
 - T: iteração
 - β_1 : fator de decréscimo do momento
 - β_2 : fator de decréscimo do ajuste

- Passos
 - 1, 2, 5: semelhantes à otimização com momento e RMSProp
 - 3, 4: detalhe técnico para impulsionar $\mathbf{m}$ e $\mathbf{s}$
 - ao serem inicializados com zero, haveria tendência para ser 0 no início do treino
- Valores típicos
 - β_1 : 0.9
 - β_2 : 0.999
 - ε : 10^{-7}
 - η : 0.001

Variantes

AdaMax

1. Substitui passo 2 por
 $s \leftarrow \max(\beta_2 s, \nabla_{\theta} J(\theta))$

2. Remove passo 4

3. Substitui passo 5 por
 $\theta \leftarrow \theta + \eta \mathbf{m} \oslash \sqrt{\mathbf{s}}$

- Pode ser mais estável que Adam, mas depende do problema
- Em geral Adam tem melhor desempenho

Nadam

- Adam com "truque" Nestorov
 - mede o gradiente não na posição θ , mas um pouco mais à frente
- Muitas vezes converge mais rápido

Comparação

Class	Convergence speed	Convergence quality
SGD	*	***
SGD(momentum=...)	**	***
SGD(momentum=..., nesterov=True)	**	***
Adagrad	***	* (stops too early)
RMSprop	***	** or ***
Adam	***	** or ***
Nadam	***	** or ***
AdaMax	***	** or ***

Keras

- Gradiente com momento

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9)
```

- NAG

```
optimizer = keras.optimizers.SGD(lr=0.001, momentum=0.9, nesterov=True)
```

- Adagrad

- embora exista não deve ser utilizado

- RMSProp

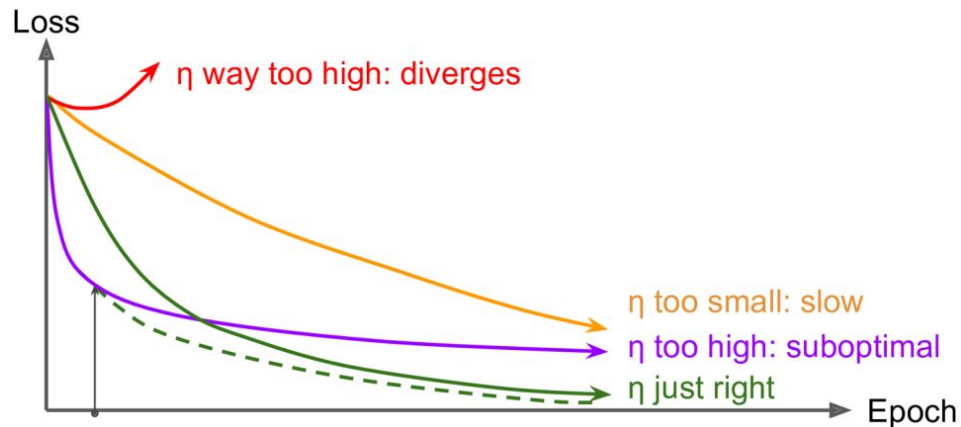
```
optimizer = keras.optimizers.RMSprop(lr=0.001, rho=0.9)
```

- Adam

```
optimizer = keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999)
```

Taxa de aprendizagem

- Encontrar a taxa de aprendizagem adequada é muito importante!
 - muito alta
 - treino diverge
 - muito baixa
 - converge para o ótimo mas demora muito tempo
 - um pouco alta
 - faz grandes progressos no início, mas saltita à volta do ótimo sem estabilizar



Taxa de aprendizagem constante

1. treinar o modelo algumas centenas de iterações
2. aumentar a taxa exponencialmente começando num valor pequeno até um valor muito grande
3. observar a **curva de aprendizagem** e escolher um valor pouco menor que aquele onde a curva volta a subir
4. re-inicializar o treino com esse valor

Taxa de aprendizagem variável

Power scheduling

$$\eta(t) = \frac{\eta_0}{(1 + t/s)^c}$$

- Variação
 - s passos depois: $\eta = \eta_0/2$
 - s passos depois: $\eta = \eta_0/3$
 - ...
 - reduções cada vez mais pequenas
- hiperparâmetros: η_0, c, s

Exponential scheduling

$$\eta(t) = \eta_0 0.1^{t/s}$$

- Variação
 - cai um fator de 10 em cada s passos
- hiperparâmetros: η_0, s

Piecewise constant scheduling

- Utiliza taxa constante por um n° fixo de épocas; depois usa outra menor por outro número de épocas, ...
- exemplo
 - 5 primeiras épocas: $\eta=0.1$
 - 50 épocas seguintes: $\eta=0.001$
 - ...
- implica encontrar a sequência de taxas e o n° de épocas corretos

Performance decay

- Mede o erro de validação em cada N passos e reduz a taxa por um fator λ quando o erro deixa de diminuir

1cycle scheduling

- processo
 - a. aumenta linearmente a taxa inicial η_0 até η_1 a meio do treino
 - b. diminui depois até η_0 na segunda metade
 - c. termina as últimas épocas com um valor menor várias ordens de magnitude
 - η_1 : mesma aproximação da pesquisa da taxa de aprendizagem constante
 - η_0 : aproximadamente 10 vezes menor
- usar estratégia semelhante para o momento
 - a. começar com valor alto (0.95), diminuir linearmente (0.85) e voltar a aumentar

Melhores estratégias

- Estudos mostram que as seguintes estratégias aceleram a convergência
 - exponential decay
 - performance scheduling
 - 1cycle scheduling

Regularização

Redes neuronais profundas

- Possuem dezenas de milhares de parâmetros (se não milhões)
 - imensos graus de liberdade
 - ajustam-se a uma grande variedade de problemas complexos
 - propensas a sobre-ajustamento ao conj treino
- Necessidade de regularização!
 - *early stopping*
 - batch normalization (embora pensado para resolver o problema dos gradientes)

Regularização ℓ_1 e ℓ_2

- ℓ_2
 - restringir os pesos
- ℓ_1
 - modelo esperso
- keras
 - adicionar um regularizador em cada camada
 - `keras.regularizers.l1()`
 - `keras.regularizers.l2()`
 - `keras.regularizers.l1_l2()`

```
layer = keras.layers.Dense(100, activation="elu",  
                             kernel_initializer="he_normal",  
                             kernel_regularizer=keras.regularizers.l2(0.01))
```

Keras: refatorização do código

- Usar ciclos
- `functools.partial()`

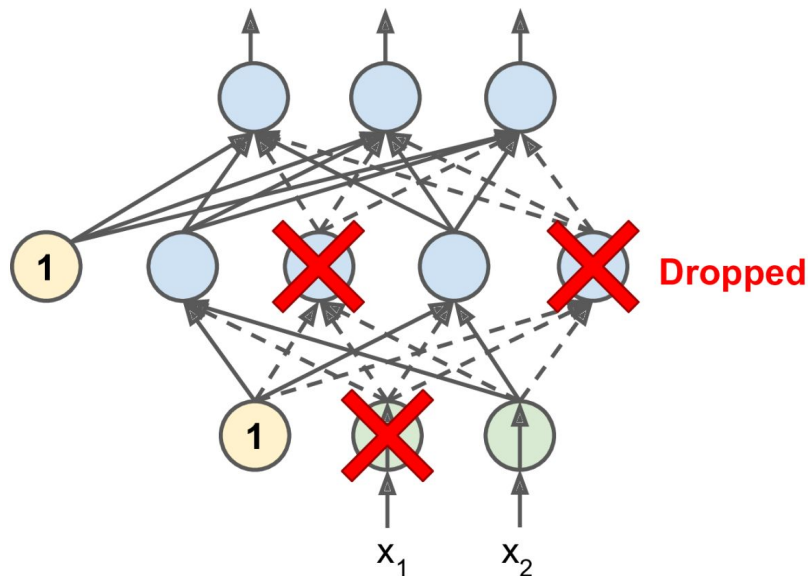
```
from functools import partial
```

```
RegularizedDense = partial(keras.layers.Dense,  
                           activation="elu",  
                           kernel_initializer="he_normal",  
                           kernel_regularizer=keras.regularizers.l2(0.01))
```

```
model = keras.models.Sequential([  
    keras.layers.Flatten(input_shape=[28, 28]),  
    RegularizedDense(300),  
    RegularizedDense(100),  
    RegularizedDense(10, activation="softmax",  
                    kernel_initializer="glorot_uniform")  
])
```

Dropout

- Proposto em 2012, Geoffrey Hinton
- Algoritmo
 - em cada passo de **treino** qualquer neurónio (exceto os de saída) tem uma probabilidade p de ser **esquecido** (ignorado)
- taxa de *dropout*
 - p
 - entre 10% e 50%



Neurónios e dropout

- neurónios treinados com dropout
 - não se podem co-adaptar com os vizinhos
 - têm de ser o mais úteis possível sozinhos...
 - não podem depender demasiado de alguns neurónios
 - são menos sensíveis a pequenas alterações das entradas
- rede mais robusta e que generaliza melhor
- em cada passo de treino é gerada uma nova rede (treinada por um passo)
 - rede resultante pode ser vista como um comité de todas as redes menores

keep probability

- 1-p
 - Depois do treino é necessário compensar o peso de cada entrada (manutenção da amplitude do sinal)
 - multiplicar cada entrada por (1-p)
 - dividir cada saída por (1-p)

keras

- `keras.layers.Dropout()`
- treino
 - esquece entradas com probabilidade p
 - divide as restantes por (1-p)
- teste
 - não faz nada

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
```

Ajustamento e dropout

- Comparar a perda sobre os conjuntos de treino e validação durante o treino pode ser enganador!
 - Deve apenas avaliar a perda sobre o teste depois do treino (sem dropout...)
- Sobre-ajustamento
 - Aumentar dropout
- Sub-ajustamento
 - Diminuir dropout
- Tamanho das camadas
 - Camadas maiores -> maior dropout
 - Camadas mais pequenas -> menor dropout

Desvantagens

- torna a convergência muito mais lenta!

alpha dropout

- Variante para redes auto-normalizadas
- Redes auto-normalizadas
 - baseadas na função de ativação SELU
 - preservam a média e desvio padrão das suas entradas

Monte Carlo (MC) Dropout

- Técnica que melhora o desempenho de um modelo com dropout sem necessidade de o re-treinar ou modificar
 - prevê o conj de teste n vezes com dropout e faz a média
 - mais confiável que o resultado de uma única previsão com o dropout desativado
- Exemplo
 - n=100
 - training=True
 - o dropout mantém-se ativo
 - cada previsão é algo diferente

```
y_probas = np.stack([model(X_test_scaled, training=True)  
                      for sample in range(100)])  
y_proba = y_probas.mean(axis=0)
```

Modelo com camadas de comportamento específico

- por exemplo, batch normalization
 - não se deve forçar o modo de treino (altera os pesos dessas camadas)
 - deve-se substituir a camada de dropout por uma que ative o treino

```
class MCDropout(keras.layers.Dropout):  
    def call(self, inputs):  
        return super().call(inputs, training=True)
```

Regularização Max-Norm

- Para cada neurónio
 - restringe os pesos de entrada para uma norma máxima r
 - $\|w\|_2 \leq r$
- r
 - hiperparâmetro
 - menor $r \rightarrow$ maior regularização
- Ajuda no problema dos gradientes instáveis

- keras

```
keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal",  
                    kernel_constraint=keras.constraints.max_norm(1.))
```

Guidelines práticas

Estratégias para diminuir o tempo de treino

- Aplicar uma boa estratégia de inicialização de pesos
- Escolher uma boa função de ativação
- Usar *batch normalization*
- Reutilizar (parte de) uma rede pré-treinada
- Utilizar um otimizador mais rápido que descida do gradiente

Que técnicas usar?

- Não existe um consenso claro
 - **Depende da tarefa!**
- Na prática e na maioria dos casos, a seguinte configuração funciona bem
 - Inicialização de pesos: He
 - Função de ativação: ELU
 - Normalização: Batch Norm se profunda; sem normalização se superficial
 - Regularização: Early stopping (+ ℓ_2 se necessário)
 - Otimizador: momentum (ou RMSProp ou Nadam)
 - Escalonamento da taxa de aprendizagem: 1 cycle

- Rede de camadas densas -> pode auto-normalizar
 - Inicialização de pesos: LeCun
 - Função de ativação: SELU
 - Normalização: sem normalização
 - Regularização: alpha dropout (se necessário)
 - Otimizador: momentum (ou RMSProp ou Nadam)
 - Escalonamento da taxa de aprendizagem: 1 cycle

Bibliografia

Capítulo 11

*Hands-On Machine Learning with
Scikit-Learn, Keras & TensorFlow*
